Benchmark Comparativo entre Ariel OS e FreeRTOS em Sistemas de Tempo Real Restritos

Experiência Criativa: Projeto Transformador II
Bacharelado em Ciência da Computação – PUCPR

Turma A – Equipe 4
Bruno Moschetta, Daniel Lima, Isabella Conceição, Vinícius Pan
bruno.moschetta, l.pereira15, isabella.lucena, vinicius.pan@pucpr.edu.br

I. Descrição do Projeto

O crescimento acelerado de sistemas embarcados e dispositivos de Internet das Coisas (IoT) tem impulsionado a adoção de sistemas operacionais de tempo real (RTOS) capazes de operar sob fortes restrições de memória, energia e previsibilidade temporal em microcontroladores de 32 bits [1]. Nesse cenário, o FreeRTOS se consolidou como o RTOS de referência na indústria, sendo amplamente utilizado por sua maturidade, leveza e suporte extenso a arquiteturas de hardware. Entretanto, por ser implementado em C, herda as fragilidades clássicas dessa linguagem quanto ao gerenciamento manual de memória, como evidenciado por ataques de buffer overflow (return-to-libc e injeção de código) bem-sucedidos mesmo diante dos mecanismos de proteção disponíveis no FreeRTOS [2] e por estudos de injeção de falhas que mostram a sensibilidade de suas estruturas internas (TCBs, filas e variáveis globais) a corrupções que comprometem a integridade do sistema [3].

Paralelamente, a linguagem Rust vem se consolidando como alternativa para software de sistemas embarcados, combinando controle de baixo nível com segurança de memória garantida em tempo de compilação por meio de seu modelo de posse (*ownership*) características que já motivaram o surgimento de diversos RTOS e frameworks de concorrência embarcados em Rust, como RTIC, Embassy e o kernel de tempo real HarSaRK-RS, que reporta reduções expressivas de tempo de *context switch* frente a kernels equivalentes em C [4]. Nesse contexto, surge o Ariel OS, um RTOS recente, open source e escrito inteiramente em Rust, que introduz suporte a escalonamento preemptivo multicore combinado à programação assíncrona sobre arquiteturas ARM Cortex-M, RISC-V e Xtensa [1].

Embora trabalhos recentes já tenham comparado quantitativamente frameworks de concorrência em Rust e C, como a avaliação de FreeRTOS, RTIC e Embassy em métricas de latência, jitter e uso de memória [5], e estudos de caso industriais tenham indicado que Rust não impõe penalidades relevantes de desempenho ou footprint frente ao C [6], ainda há uma lacuna de avaliações empíricas comparando diretamente o Ariel OS ao FreeRTOS sob métricas de desempenho padronizadas.

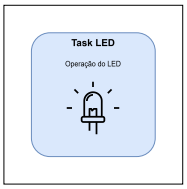
Este artigo busca preencher essa lacuna propondo uma metodologia de *benchmarking* do Ariel OS e do FreeRTOS na ESP32-WROOM-32, buscando identificar os *trade-offs* entre os dois sistemas quanto ao desempenho. Em adição, será discutido de forma qualitativa e com base na literatura, se as garantias de segurança de memória (*(memory-safety)* proporcionadas por Rust tornam o Ariel OS uma alternativa viável ao FreeRTOS.

Para esse experimento, serão desenvolvidas três aplicações de diferentes complexidades: a aplicação *blinky* (simples), a aplicação *UART-to-LED* (média), e a aplicação *IoT* (complexa). Ao fim, essas aplicações serão usadas para avaliar métricas como *footprint* de memória, *jitter* periódico, latência de interrupção e latência de *task-switch* entre os dois sistemas operacionais, permitindo comparar seu desempenho e sua previsibilidade temporal, isto é, a variação dos períodos e latências observados entre diferentes execuções.

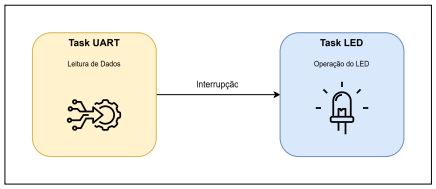
II. Materiais e Métodos

A avaliação experimental irá comparar o FreeRTOS, baseado em C, e o Ariel OS, desenvolvido em Rust, utilizando cargas de trabalho equivalentes executadas sobre a mesma plataforma de hardware. Será utilizada a ESP32-WROOM-32, baseada na arquitetura Xtensa, que permanecerá fixa durante a avaliação. Trabalhos anteriores demonstram a execução do Ariel OS em diferentes arquiteturas de microcontroladores [1], incluindo plataformas da família ESP. A equipe dispõe da placa necessária e possui acesso aos ambientes de desenvolvimento e *toolchains* utilizados para compilação, gravação e execução das implementações em FreeRTOS e Ariel OS.

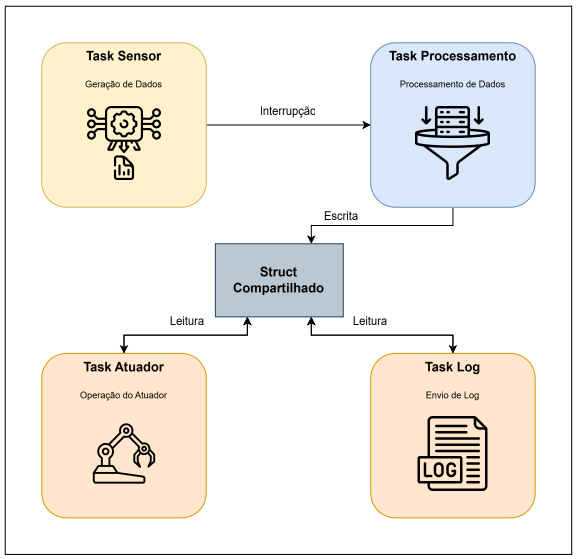
Serão desenvolvidas três aplicações de complexidade crescente, cuja estrutura é apresentada na Fig. 1. A primeira, *Blinky* (Fig. 1a), consistirá em uma única tarefa periódica responsável por alternar o estado de um LED. A segunda, *UART-to-LED* (Fig. 1b), realizará a recepção de dados pela interface UART e a sinalização da tarefa responsável pela alteração do LED. Por fim, a aplicação IoT (Fig. 1c) será composta pelas funções de aquisição de dados de um sensor, processamento das leituras, atuação e transmissão de logs. As implementações utilizarão os mecanismos próprios de cada RTOS, preservando a mesma funcionalidade e carga de trabalho. Essa estratégia segue avaliações anteriores que empregam aplicações equivalentes de complexidade crescente



(a) Blinky



(b) UART-to-LED



(c) Aplicação IoT

Figura 1: Estrutura das aplicações utilizadas na avaliação experimental.

para comparar FreeRTOS e *frameworks* embarcados em Rust [5], [6].

As métricas avaliadas serão *footprint* de memória Flash e RAM, *jitter* periódico, latência de interrupção e latência de troca de tarefa. O *footprint* será obtido a partir dos binários gerados por cada implementação. O jitter será medido a partir da diferença entre o período esperado e o período efetivamente observado em tarefas periódicas. A latência de interrupção corresponderá ao intervalo entre a geração de um evento e o início de sua rotina de tratamento, enquanto a latência de task-switch medirá o intervalo entre a sinalização de uma tarefa e o início de sua execução. Essas métricas são utilizadas em avaliações recentes de frameworks de concorrência para sistemas embarcados [5].

Antes da coleta definitiva, cada configuração experimental será submetida a 30 execuções piloto (uma quantidade reduzida), utilizada para validar o procedimento de coleta e observar a variabilidade das métricas temporais. A partir do comportamento observado nessas execuções, será definida a quantidade de repetições necessária para a coleta definitiva, buscando manter uma amostra representativa para cada configuração. Os resultados serão analisados por meio de estatísticas descritivas, incluindo média, mediana, desvio-padrão e percentis, permitindo comparar tanto o desempenho médio quanto a previsibilidade temporal dos dois sistemas.

Tabela I: Resumo dos materiais e métodos

Item	Descrição
RTOS	FreeRTOS e Ariel OS
Hardware	ESP32-WROOM-32 (Xtensa)
Aplicações	Blinky, UART-to-LED e aplicação IoT
Métricas	Flash, RAM, jitter, latência de interrupção e task-switch
Análise	Avaliação experimental de desempenho e previsibilidade temporal

Este projeto não utiliza um conjunto de dados externo. Os dados experimentais serão produzidos pelas próprias execuções dos *benchmarks*, sendo compostos pelas medições obtidas para cada RTOS e aplicação. As versões dos sistemas operacionais, compiladores, parâmetros de compilação e configurações utilizadas serão registradas para garantir a reprodutibilidade dos experimentos.

A tabela I resume os principais materiais, aplicações, métricas e procedimentos adotados na avaliação experimental.

III. Estágios do Projeto e Marcos Observáveis

O desenvolvimento será organizado em seis etapas progressivas com marcos verificáveis no repositório do projeto. Algumas implementações por não serem dependentes entre si podem ocorrer em paralelo, com revisão cruzada entre os integrantes para manter a equivalência funcional dos experimentos.

1) Validação da plataforma e implementação do Blinky A ESP32-WROOM-32 será utilizada como plataforma comum aos experimentos. Inicialmente, serão configuradas e validadas as toolchains necessárias para FreeRTOS e Ariel OS. Em seguida, a aplicação *Blinky* será implementada em conjunto nos dois ambientes, servindo como teste inicial de compilação, gravação, execução e acesso aos periféricos. Essa etapa permite também que os integrantes conheçam a estrutura básica dos dois RTOS antes da divisão de implementações. O marco de conclusão será a execução funcional do *Blinky* em FreeRTOS e Ariel OS na mesma plataforma, acompanhada de código-fonte, instruções e logs de execução no repositório.

2) Implementação do UART-to-LED Após validação inicial, será desenvolvida a aplicação *UART-to-LED*. Cada integrante poderá assumir inicialmente a implementação em um dos RTOS, preservando a mesma funcionalidade e os mesmos parâmetros de execução. Ao final, as implementações serão revisadas e validadas de forma cruzada. O marco será a execução da aplicação nos dois RTOS, comprovando a recepção do mesmo comando pela UART e a correspondente alteração do estado do LED, acompanhada dos códigos-fonte e evidências de execução no repositório.

3) Implementação da aplicação IoT Será implementada a aplicação de maior complexidade, composta por aquisição de dados de sensor, processamento, atuação e transmissão de logs. As versões em FreeRTOS e Ariel OS deverão preservar a mesma funcionalidade e parâmetros equivalentes de execução. O marco será a execução, nos dois RTOS, do fluxo completo de aquisição, processamento,

atuação e transmissão, seguida dos códigos-fonte e evidência de execução.

4) Instrumentação e validação das medições
Serão incorporados os mecanismos necessários para obtenção das métricas de Flash, RAM, *jitter*, latência de interrupção e *task-switch*. Serão realizadas execuções para validar procedimento de coleta, formato dos dados e consistência das medições antes da execução definitiva. O marco será a existência de um procedimento documentado e automatizável, capaz de produzir as mesmas métricas para ambos os RTOS sob configurações equivalentes.

5) Execução e análise dos experimentos
Os *benchmarks* serão executados repetidamente sob configurações equivalentes de software e hardware. As medições serão armazenadas em arquivos estruturados e analisadas por meio das estatísticas definidas na Seção II. O marco de conclusão será a obtenção de todas as medições previstas no protocolo experimental FreeRTOS e Ariel OS, acompanhadas de tabelas e gráficos comparativos.

6) Consolidação dos resultados
Os resultados serão interpretados em relação à literatura, buscando identificar os *trade-offs* entre FreeRTOS e Ariel OS. O marco de conclusão será evidenciada pela organização dos artefatos experimentais no repositório, pela versão final do artigo científico e pôster de apresentação do projeto, incorporando análise dos resultados e suas possíveis limitações.

IV. Cronograma

O cronograma do projeto foi organizado de forma progressiva, acompanhando as etapas de desenvolvimento definidas na Seção III. As atividades iniciais de revisão bibliográfica, definição do escopo e familiarização com as linguagens, RTOS e ferramentas de desenvolvimento foram realizadas durante o período de planejamento. O desenvolvimento experimental seguirá pela validação da plataforma, implementação das aplicações Blinky, *UART-to-LED* e IoT, instrumentação das métricas, execução dos *benchmarks* e análise dos resultados.

As atividades foram distribuídas considerando a possibilidade de trabalho paralelo entre os integrantes e os principais marcos definidos no plano de ensino, incluindo a entrega de codificação, metodologia implementada e resultados em 31/10, a entrega do artigo científico e pôster em 09/11 e a apresentação na *Poster Session* em 17/11. A Tabela II apresenta a distribuição prevista das atividades ao longo do semestre.

Tabela II: Cronograma de desenvolvimento do projeto.

Atividade	04/08–17/08	18/08–31/08	01/09–14/09	15/09–28/09	29/09–12/10	13/10–31/10	01/11–09/11	10/11–17/11
Revisão bibliográfica e definição do escopo	X	X						
Aprendizagem das linguagens, RTOS e dinâmica de desenvolvimento	X	X						
Validação da plataforma e Blinky			X					
Implementação do UART-to-LED			X	X				
Implementação da aplicação IoT				X	X			
Instrumentação e execuções piloto					X	X		
Execução e análise dos experimentos						X	X	
Consolidação dos resultados							X	
Artigo científico e pôster						X	X	
Preparação para a Poster Session							X	

V. Baseline de Implementação

O código-fonte e os artefatos da *baseline* inicial estão disponíveis no repositório público do projeto https://github.com/Vinokaa/FreeRTOS-ArielOS-Benchmark.

Como ponto de partida, a equipe validou a execução da aplicação Blinky na ESP32-WROOM-32 usando o FreeRTOS, bem como o Ariel OS. Essas implementações permitiram validar a configuração das *toolchains*, o processo de compilação e gravação e o acesso aos periféricos. O repositório contém os códigos-fonte, arquivos de configuração, instruções de compilação e gravação e evidências de execução da *baseline* atual.

Referências

[1] E. Frank, K. Schleiser, R. Fouquet, K. Zandberg, C. Amsüss, and E. Baccelli, "Ariel OS: An embedded Rust operating system for networked sensors & multi-core microcontrollers," in *2025 21st International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*. IEEE, 2025, pp. 241–245.

[2] G. Mullen and L. Meany, "Assessment of buffer overflow based attacks on an IoT operating system," in *2019 Global IoT Summit (GIoTS)*. Aarhus, Denmark: IEEE, 2019, pp. 1–6.

[3] A. Bosio, M. Rebaudengo, and A. Savino, "Reliability assessment of FreeRTOS in embedded systems," in *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks - Supplemental Volume (DSN-S)*. IEEE, 2022.

[4] N. Gupta, G. Banda, N. Hubballi, and K. V. Srinivas, "A hard real-time kernel for CPIoT systems with safety features in Rust," *IEEE Access*, vol. 13, pp. 116 803–116 820, 2025.

[5] G. Wang, "Concurrency frameworks for embedded systems: A comparative evaluation of FreeRTOS, RTIC, and Embassy," *IET Computers & Digital Techniques*, vol. 2026, p. 7881841, 2026.

[6] B. Thapa, D. Alfonso, L. Bini, L. Mapelli, K. Schleiser, R. Fouquet, and E. Baccelli, "Embedded Rust or C firmware? lessons from an industrial microcontroller use case with Ariel OS," in *Proceedings of IEEE DCOSS-IoT, Workshop on IoT Applications and Industry 5.0 (IoTI5)*. IEEE, June 2026.